

CESS-UFF / SGIMP

Avaliação prática do processo seletivo

Sistema de sinalização com ESP32, LEDs, botão e OLED SSD1306

Relatório técnico resumido

Candidato:	<i>preencher antes da entrega</i>
Plataforma de simulação:	Wokwi
Linguagem:	MicroPython
Placa-alvo:	Espressif ESP32-DevKitC V4

5 de agosto de 2026

Sumário

1	Introdução	2
2	Requisitos e definições do projeto	2
2.1	Requisitos funcionais	2
2.2	Definições de hardware	2
2.3	Entrada do botão e antirrepique	3
2.4	Interface do mostrador	3
3	Desenvolvimento do programa main.py	3
3.1	Estrutura geral	3
3.2	Assincronismo cooperativo	4
3.3	Controle do LED vermelho	5
3.4	Controle do botão, LED verde e mensagens	5
3.5	Atualização dinâmica do OLED	5
4	Estratégia de verificação e testes isolados	5
5	Circuito simulado	6
6	Conclusão	6
	Referências eletrônicas	7

1 Introdução

Este projeto integra uma avaliação prática do processo seletivo **CESS-UFF / SGIMP**. A proposta reúne conteúdos de instrumentação, eletrônica e lógica de programação por meio da simulação de um sistema embarcado baseado em ESP32, dois diodos emissores de luz (LEDs), um botão pulsador e um mostrador OLED controlado pelo circuito integrado SSD1306.

A simulação executável está publicada no Wokwi em <https://wokwi.com/projects/471528241540407297>. O código-fonte, o diagrama do circuito, a documentação e o histórico de versões estão disponíveis no repositório GitHub em <https://github.com/Argus-Kepheus/cess-uff>.

O Wokwi foi adotado como plataforma principal por oferecer suporte nativo ao ESP32 com MicroPython, edição visual do circuito, arquivo de conexão `diagram.json`, monitor serial e compartilhamento por endereço eletrônico. O repositório GitHub complementa a simulação ao fornecer rastreabilidade, controle de versões e uma estrutura adequada ao desenvolvimento no Visual Studio Code.

2 Requisitos e definições do projeto

2.1 Requisitos funcionais

O sistema deve satisfazer os seguintes requisitos:

1. O LED vermelho, conectado ao GPIO 2, deve alternar entre ligado e desligado a cada 500 ms, continuamente e sem bloquear as demais funções.
2. O LED verde, conectado ao GPIO 4, deve permanecer apagado com o botão solto e acender enquanto o botão estiver pressionado.
3. O botão pulsador normalmente aberto deve ser conectado ao GPIO 17 e interpretado como nível lógico alto (*HIGH*) quando pressionado.
4. O mostrador OLED SSD1306 deve utilizar comunicação I²C, com GPIO 25 como SCL e GPIO 16 como SDA.
5. Com o botão solto, o OLED deve exibir exatamente a mensagem **Boa sorte!!**.
6. Com o botão pressionado, o OLED deve exibir exatamente a mensagem **Consegui!**.
7. A mensagem e o LED verde devem ser atualizados quando o estado estável do botão mudar.

2.2 Definições de hardware

A placa selecionada foi a **Espressif ESP32-DevKitC V4**, representada no Wokwi pelo componente `board-esp32-devkit-c-v4`. A especificação original exige apenas “MicroPython para ESP32”; portanto, a escolha da placa não decorre de uma necessidade exclusiva de desempenho. Ela foi assumida por ser uma placa oficial da Espressif, documentada, identificada de forma inequívoca no simulador e capaz de disponibilizar todos os GPIOs requeridos.

Para eventual montagem física, recomenda-se uma ESP32-DevKitC V4 equipada com módulo ESP32-WROOM, preferencialmente ESP32-WROOM-32E. Versões com módulo WROVER devem ser evitadas sem revisão do projeto, pois GPIO 16 e GPIO 17 podem ser reservados para a memória PSRAM.

Tabela 1: Pinagem específica do projeto.

Função	GPIO	Identificador no Wokwi	Identificador no código
LED vermelho	2	red-led	red_led
LED verde	4	green-led	green_led
Botão pulsador	17	push-button	push_button
OLED, dados I ² C	16	oled-display:SDA	OLED_SDA_PIN
OLED, relógio I ² C	25	oled-display:SCL	OLED_SCL_PIN

A numeração utilizada corresponde aos números dos GPIOs do microcontrolador e não à posição sequencial dos terminais físicos da placa. Os dois LEDs possuem resistores limitadores de 220 Ω , e todos os componentes compartilham a mesma referência de terra.

2.3 Entrada do botão e antirrepique

O GPIO 17 é configurado como entrada com resistor interno de redução (*pull-down*). Dessa forma, o terminal permanece em nível baixo com o botão aberto e passa ao nível alto quando o contato conecta o GPIO à alimentação de 3,3 V.

Não foi acrescentado filtro RC externo. O antirrepique é realizado por programa, sem bloqueio: o nível é amostrado periodicamente e uma alteração somente é aceita quando permanece estável por aproximadamente 30 ms. Essa estratégia é suficiente para a aplicação e evita oscilações do LED verde e atualizações repetidas do OLED.

2.4 Interface do mostrador

A utilização de GPIO 25 para SCL e GPIO 16 para SDA é uma **predefinição do projeto**. Não se trata da escolha dos pinos mais usuais do ESP32, razão pela qual o mapeamento é declarado explicitamente no código, no diagrama e na documentação.

A interface I²C utiliza apenas dois sinais de comunicação e é adequada às mensagens curtas e estáticas desta aplicação. Uma implementação SPI poderia alcançar maior taxa de transferência, porém exigiria mais sinais de controle e não proporcionaria vantagem relevante para as atualizações pouco frequentes deste projeto. O endereço adotado para o SSD1306 é 0x3C, com resolução de 128 por 64 pixels.

3 Desenvolvimento do programa main.py

3.1 Estrutura geral

O arquivo `main.py` concentra a inicialização do hardware e a coordenação das tarefas. Os nomes dos componentes e das variáveis seguem convenções distintas: identificadores do Wokwi usam hífen, enquanto identificadores Python usam sublinhado. As constantes de pino permanecem em letras maiúsculas.

Uma organização representativa da aplicação é mostrada a seguir:

Listing 1: Estrutura lógica resumida do programa.

```
1 async def blink_red_led():
2     """Toggle the red LED every 500 milliseconds."""
3     while True:
4         red_led.value(not red_led.value())
5         await asyncio.sleep_ms(500)
6
7
8 async def monitor_push_button():
9     """Read, debounce, and apply stable button-state changes."""
10    ...
11
12
13 async def main():
14     asyncio.create_task(blink_red_led())
15     asyncio.create_task(monitor_push_button())
16
17     while True:
18         await asyncio.sleep_ms(1000)
19
20
21 asyncio.run(main())
```

O trecho é uma síntese da arquitetura; o código completo e executável permanece no repositório.

3.2 Assincronismo cooperativo

A biblioteca `asyncio` do MicroPython foi escolhida como arquitetura exclusiva do projeto. Em vez de um superlaço que verifica manualmente diversos temporizadores, cada função independente pode ser organizada como uma corrotina. A instrução `await asyncio.sleep_ms(...)` suspende apenas a tarefa atual e devolve o controle ao escalonador.

Essa decisão foi tomada pelos seguintes motivos:

- separação clara entre o piscar do LED vermelho, a leitura do botão e a atualização das saídas;
- menor acoplamento entre funcionalidades;
- facilidade de manutenção e expansão com novos sensores ou atuadores;
- prevenção de congelamentos causados por `time.sleep()` no fluxo principal;
- melhor tolerância às pequenas latências introduzidas pela transmissão do quadro de imagem do OLED pelo barramento I²C.

O assincronismo adotado é cooperativo. Portanto, as tarefas devem alcançar pontos de espera com frequência e evitar rotinas longas ou bloqueantes.

3.3 Controle do LED vermelho

O LED vermelho opera em tarefa própria e alterna seu nível lógico a cada 500 ms. O ciclo completo, composto por 500 ms ligado e 500 ms desligado, dura aproximadamente um segundo. Essa tarefa não depende do botão, do LED verde ou do OLED.

O uso de `time.sleep_ms(500)` foi aceito apenas em um programa temporário de diagnóstico isolado. No programa final, a espera é assíncrona para que a leitura do botão continue ocorrendo durante o intervalo entre as alternâncias.

3.4 Controle do botão, LED verde e mensagens

A tarefa de entrada realiza amostragem periódica, antirrepique e detecção de mudança de estado estável. Quando uma transição é confirmada, uma única rotina aplica o estado às duas saídas dependentes:

Tabela 2: Tabela de estados do botão, LED verde e OLED.

Estado estável do botão	LED verde	Mensagem do OLED
Solto (LOW)	Apagado	Boa sorte!!
Pressionado (HIGH)	Aceso	Conseguí!

A sincronização do LED verde e do OLED a partir do mesmo estado estável evita divergências entre a indicação luminosa e a mensagem apresentada.

3.5 Atualização dinâmica do OLED

O mostrador não é redesenhado continuamente. O quadro de imagem é transmitido na inicialização e somente quando o estado confirmado do botão muda. Essa atualização orientada por eventos reduz:

- tráfego desnecessário no barramento I²C;
- tempo de processamento;
- repetição de escritas no controlador SSD1306;
- cintilação visual do mostrador;
- interferência temporal nas demais tarefas.

A escolha é especialmente apropriada porque as mensagens permanecem estáticas entre uma transição e outra.

4 Estratégia de verificação e testes isolados

O desenvolvimento foi acompanhado por testes progressivos. A separação dos subsistemas permitiu distinguir erros de ligação, inicialização e lógica de aplicação.

1. **Teste isolado do LED vermelho:** utilizou um laço bloqueante mínimo somente para diagnóstico. O GPIO 2 foi alternado a cada 500 ms e o estado foi registrado no

monitor serial. O objetivo foi confirmar placa, pino, resistor, polaridade e retorno ao terra sem interferência das demais funções.

2. **Teste básico do OLED:** inicializou o barramento I²C em GPIO 25 e GPIO 16, realizou `i2c.scan()` e confirmou a detecção do endereço decimal 60, equivalente a 0x3C. Em seguida, apresentou uma mensagem simples.
3. **Diagnóstico gráfico do SSD1306:** exercitou todos os pixels ligados e desligados, padrões quadriculados complementares, varreduras horizontais e verticais, grade, pixels individuais, linhas, retângulos, áreas preenchidas, texto, inversão, contraste, desligamento, religamento e deslocamento do quadro.
4. **Reinício do diagnóstico pelo botão:** o teste completo do OLED foi adaptado para reiniciar com um pulso no GPIO 17, incluindo antirrepique e detecção de borda de subida. Essa etapa verificou simultaneamente entrada digital, resistor interno de redução e execução assíncrona.
5. **Integração final:** após a validação individual, foram reunidos o piscar independente do LED vermelho, o LED verde comandado pelo botão e a atualização dinâmica das mensagens do OLED.

A sequência de testes reduziu o espaço de busca de falhas: se um subsistema simples não funcionasse, o problema poderia ser tratado antes da integração com o restante da aplicação.

5 Circuito simulado

A Figura 1 apresenta o estado atual do circuito no Wokwi. As cores dos condutores foram usadas para facilitar a inspeção visual: vermelho para alimentação e LED vermelho, preto para terra, verde para o LED verde, azul para o botão, amarelo para SCL e ciano para SDA.

A disposição gráfica dos fios pode ser ajustada sem modificar o funcionamento, desde que as extremidades elétricas permaneçam conectadas aos mesmos terminais. O arquivo `diagram.json` registra tanto as conexões quanto as coordenadas visuais e as instruções de roteamento dos condutores.

6 Conclusão

O projeto atende aos requisitos propostos por meio de uma implementação modular e reproduzível. A escolha da ESP32-DevKitC V4 reduz ambiguidades de pinagem, enquanto o uso explícito dos GPIOs preserva a correspondência entre código, documentação e circuito. A comunicação I²C satisfaz adequadamente a necessidade de exibição das mensagens e mantém baixo o número de conexões.

A arquitetura assíncrona permite que o LED vermelho continue piscando enquanto o botão é monitorado e o mostrador é atualizado. O antirrepique por programa elimina a necessidade de filtro externo nesta aplicação, e a atualização do OLED apenas em mudanças de estado reduz processamento e cintilação. Por fim, os testes isolados e progressivos forneceram evidências de funcionamento de cada subsistema antes da integração.

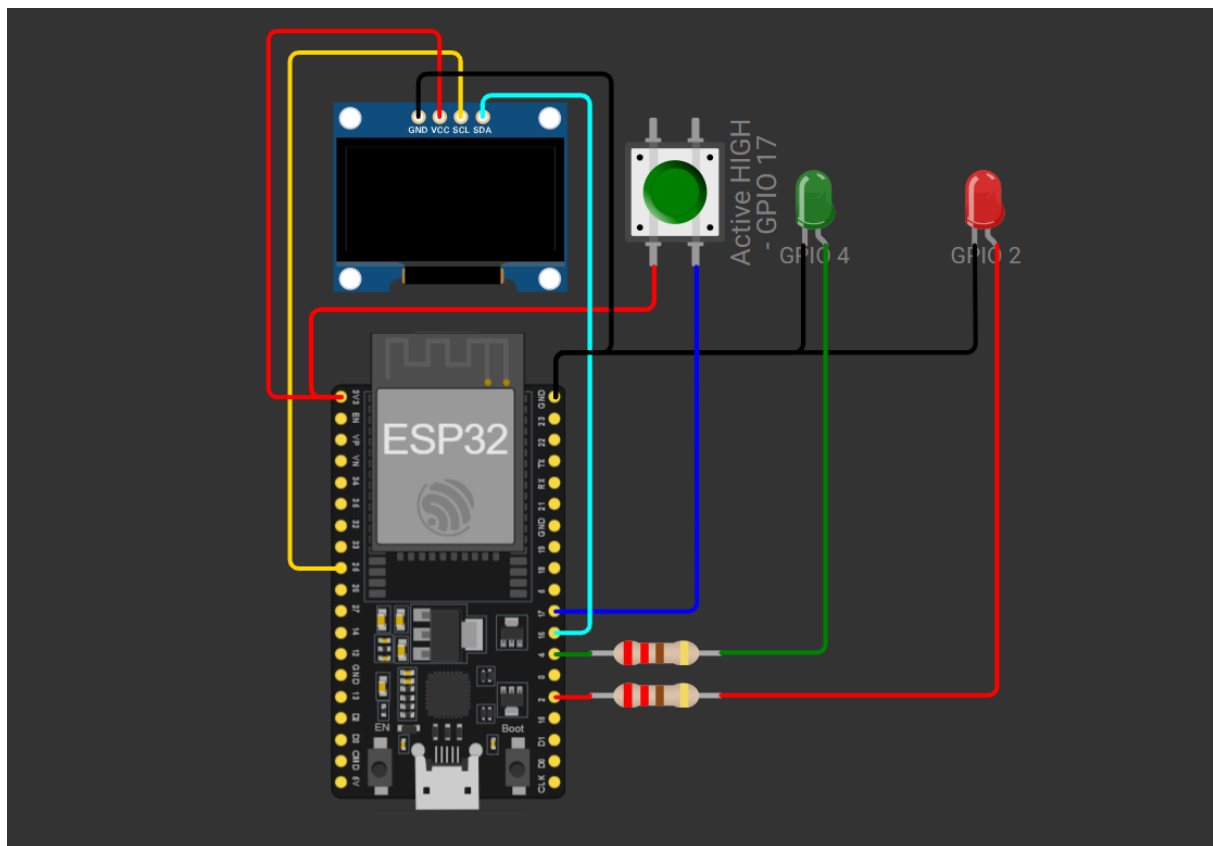


Figura 1: Circuito do projeto no Wokwi: ESP32-DevKitC V4, OLED SSD1306, botão e dois LEDs.

A entrega é complementada por dois meios de acesso: a simulação pública no Wokwi e o repositório versionado no GitHub. Em conjunto, eles possibilitam executar, inspecionar, reproduzir e evoluir o projeto.

Referências eletrônicas

- Projeto no Wokwi: <https://wokwi.com/projects/471528241540407297>
- Repositório no GitHub: <https://github.com/Argus-Kepheus/cess-uff>
- Guia da ESP32-DevKitC V4: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html
- Referência rápida do MicroPython para ESP32: <https://docs.micropython.org/en/latest/esp32/quickref.html>
- Biblioteca `asyncio` do MicroPython: <https://docs.micropython.org/en/latest/library/asyncio.html>
- Componente SSD1306 do Wokwi: <https://docs.wokwi.com/parts/board-ssd1306>